

Projet L3 : PacMan en réseau

Thomas Coutant | Benoît Litampha | Auriane Peter

Université Marie & Louis Pasteur, UFR Sciences et Techniques

Licence CMI Informatique – 3^{ème} année
2025–2026

Encadrant : Julien Bernard



Plan

- 1 Introduction
- 2 Présentation & organisation
- 3 Server
- 4 Client
- 5 Reseau
- 6 Conclusion
- 7 Démonstration

- Contexte du projet
- Objectifs
- Organisation

- **Git** pour le versionnement du projet
- **Discord** pour la communication entre les membres du groupe et avec le tuteur
- **réunions régulières** (toutes les 1 à 2 semaines) pour faire le point sur l'avancement
- Communication **quotidienne** entre les membres pour coordonner le travail



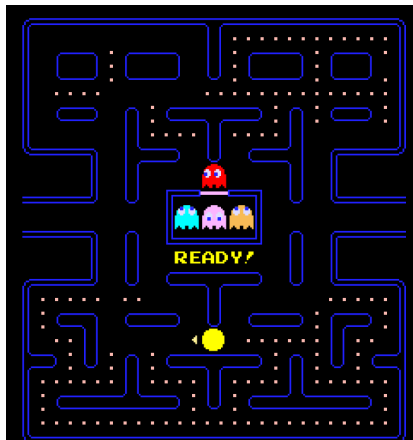
Présentation du jeu Pac-Man

Jeu d'arcade apparu dans les **années 1980**

- Pac-Man (le joueur) évolue dans un **labyrinthe**
- Objectif : manger toutes les **pac-gommes**
- Éviter les **fantômes**
- Certaines pac-gommes permettent de **manger les fantômes**

Adaptation du projet :

- 1 joueur Pac-Man
- plusieurs joueurs Fantômes
- fantômes contrôlés par **IA** si joueurs insuffisants



Jeu original Pac-Man

■ **Thomas Coutant**

- Développement du serveur
- Conception de la logique du jeu
- Prototype initial client–serveur

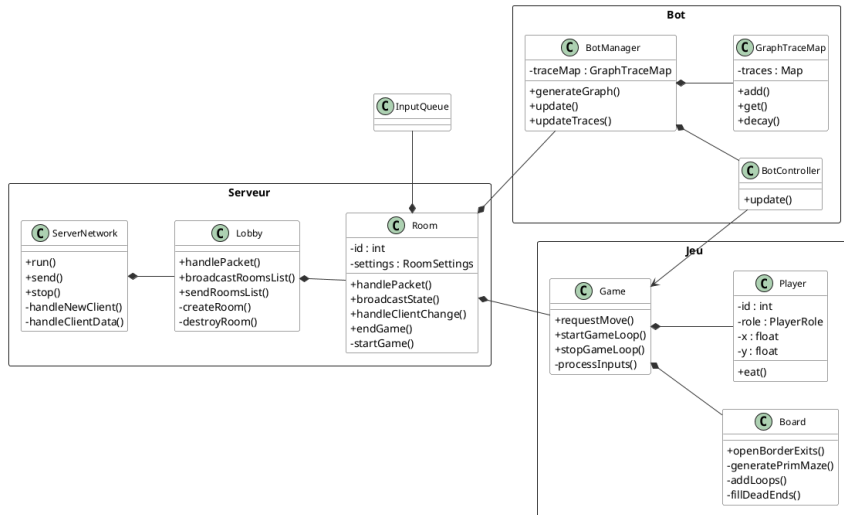
■ **Auriane Peter**

- Développement du client
- Interface et interactions joueur

■ **Benoît Litampha**

- Communication client–serveur
- Support sur client et serveur

Architecture du serveur



Client → Server → Lobby → Room → Game

Boucle de jeu

Serveur autoritaire

- Le **serveur décide de tout**
- Les **clients affichent uniquement l'état reçu**
- Permet d'**éviter la triche** et les désynchronisations

Tick serveur

- La boucle de jeu fonctionne avec des **misés à jour périodiques**
- Un **thread serveur** met à jour l'état du jeu :
 - positions des joueurs et fantômes
 - pac-gommes
 - timers
 - score et conditions de victoire

États de la partie

- Avant la partie (préparation)
- Partie en cours
- Fin de partie

IA des fantômes - Principe

Avoir des fantômes capables de s'adapter au nombre de joueurs et de produire une forme d'**intelligence collective**.

Inspiration : les fourmis

- Dans la nature, les fourmis utilisent des **phéromones** pour communiquer.
- Chaque fourmi laisse une trace que les autres peuvent suivre.

Adaptation dans le jeu

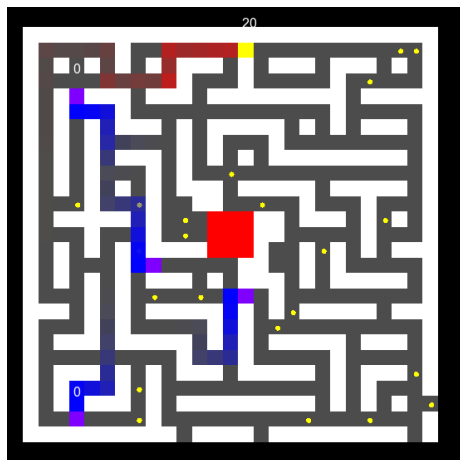
- Les entités laissent des **traces** sur le plateau.
- Chaque trace possède :
 - une **intensité** qui diminue avec le temps
 - un **propriétaire**
 - une **couleur**
- **Rouge** : Pac-Man | **Bleu** : fantômes

IA des fantômes - Prise de décision

Les fantômes choisissent leur déplacement selon le score :

$$\text{Score} = \text{Attraction} - \text{Répulsions}$$

- Les traces de **Pac-Man** créent une **attraction**
- Les traces des **fantômes** créent une **répulsion**
- La **répulsion** est plus forte pour les **traces du fantôme lui-même**

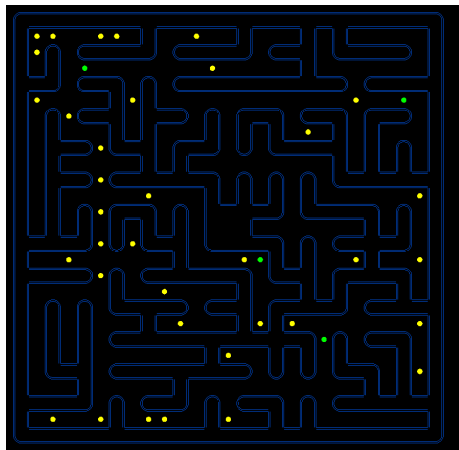


Variante de l'algorithme de Prim

- Plateau initialement rempli de murs
- Choix d'une case de départ
- Ajout des murs adjacents dans une liste
- Ouverture progressive de murs aléatoires

Résultat :

- Labyrinthe connexe
- Aucune zone isolée

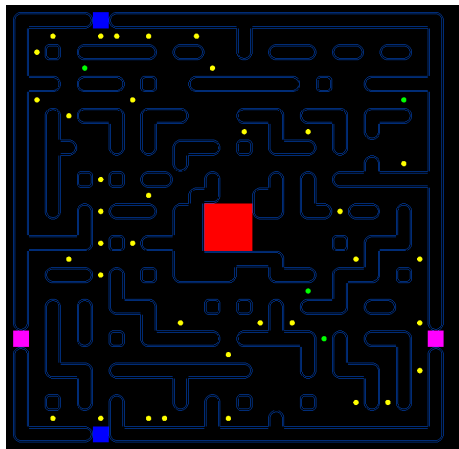


Contraintes du jeu

- Zone centrale fixe : **hut** 3×3
- Vérification de l'accès aux **coins de départ**

Améliorations

- Ajout de **cycles**
- Réduction des **culs-de-sac**
- Ajout de **portails latéraux**



TODO

TODO

TODO

Input

TODO

TODO

Communication avec socket Choix entre UDP et TCP, TCP choisit pour sa fiabilité

L'ip et le port peuvent être précisé pour le client ou le serveur en ligne de commande

Description du fonctionnement de ClientNetwork et ServerNetwork :

- Une socket de comm dans le client, socket de connexion + socket de comm pour chaque client dans le serv
- Thread pour empiler les packets

2 types de structure : Structures communes contenant les données communes entre client/server

Exemple avec PlayerData

Structures de comm utilisé pour les échanges réseau (ServerXXX = envoyé par le server, ClientXXX = envoyé par le client) Exemple avec ServerListPlayersRoom

Un Operateur | avec un type Archive, gf : :packet.is() pour sérialiser,
gf : :packet.as() pour désérialiser
Switch case avec le type du packet pour l'identifier avec l'id de la structure
Exemple avec ClientMove et ServerListPlayersRoom

Exemple concret

Un exemple concret lorsque le server broadcast la liste des joueurs avec `ServerListPlayersRoom` :

- Le serv récupère les données de chaque joueurs pour construire la structure, sérialisation avec `packet.is()`, puis envoie à chaque client
- Le client reçoit ce packet et l'empile, puis dans la boucle de jeu dépile et identifie la structure avec `gf : :ld` pour la désérialiser et afficher la liste des joueurs

Conclusion

TODO

TODO